BCT 2408 COMPUTER ARCHITECTURE

LAB 1

EARL KINUTHIA SCT 212-0067/2019

E1:

Establish the Instruction Counts

Assume the unoptimized version executes 100 instructions.

Loads/Stores: 30% of 100 = 30 instructions
Other instructions: 70 instructions

For the optimized version, the loads/stores are reduced to 2/3 of what they were, while the other instructions remain the same:

Loads/Stores: (2/3) × 30 = 20 instructions
Other instructions: 70 instructions

Thus, the total instructions in the optimized version are: 70 + 20 = 90 instructions

Consider the Clock Rates

The unoptimized version has a clock rate that is 5% higher.
If we denote the base clock rate of the optimized version as f, then the unoptimized version runs at 1.05 f.

All instructions take one clock cycle on both machines.

Calculate the Effective Execution Time

Unoptimized Version: Total cycles = 100

Since the clock is 5% faster, each cycle takes 11.05\frac{1}{1.05}1.05 of the time compared to the optimized version.

Effective time units: 100 / 1.05 = 95.24

Optimized Version:

Total cycles = 90 (because of fewer instructions)

Clock rate = f (baseline, so each cycle takes 1 time unit)

Effective time units: 90 time units

Compare the Performance

Unoptimized: 95.24 time units
Optimized: 90 time units

The optimized version takes fewer effective time units, making it faster. In fact, the optimized version's execution time is about 90 / 95.24 = 0.945 or 94.5% of the time the unoptimized version takes. This implies that the optimized version is approximately 5.5% faster.

Justification

Instruction Reduction: The optimization reduces the number of loads and stores from 30 to 20, lowering the overall instruction count by 10%.
Clock Rate Difference: Although the unoptimized version has a 5% higher clock rate, this advantage does not fully compensate for the extra instructions executed.
Overall Outcome: The reduction in instruction counts more than offsets the slower clock rate of the optimized version, leading to a net performance improvement of about 5.5%.

E2:

1. Percentage of Loads to Eliminate

Baseline

In the original load–store machine, 22.8% of the dynamic instructions are loads.

The Optimization

The idea is to replace a pair of instructions:

A load (LOAD Rx,0(Rb)) and
An add (ADD Ry,Ry,Rx) with a single register–memory add:
ADD Ry,0(Rb)

This replacement saves one instruction per occurrence.
Effect on Execution Time

Let *I* be the original total instruction count. For each replacement, we save one instruction. If a fraction *p* of the loads is eliminated by this replacement, then the total number of instructions is reduced by: Savings = $p \times (0.228 \times I)$

So, the new instruction count becomes: $I_{new} = I \times (1 - 0.228\, p)$

Clock Cycle Impact

The new instruction causes the clock period to increase by 5%. Thus, every cycle takes 1.05 times as long.

Break-even Condition

To have at least the same performance, the total execution time with the new instruction set must not exceed that of the original machine. Writing execution time as: Time = Instruction Time × Clock Cycle Time

we set up the inequality: $I \times (1 - 0.228\ p) \times 1.05 \leq I$

Canceling *I* and solving for *p*:

Rearranging:

$0.228\ p \geq 1 - 0.95238 = 0.04762$

$p \geq 0.0476 / 0.228 = 0.2088.$

Conclusion

Approximately 20.9% of the loads must be eliminated for the new machine to perform at least as well as the original.

2. A Situation Where Replacement Is Not Applicable

The replacement only works if the loaded value is used immediately in a single add instruction. Consider this sequence:

```
LOAD Rx, 0(Rb)
ADD Ry, Ry, Rx
SUB Rz, Rz, Rx
```

Explanation

The first instruction loads a value from memory into register Rx.
The second instruction uses Rx in an add operation. This pair is a candidate for replacement.
However, the third instruction also uses Rx.

If you replace the first two instructions with: ADD Ry, 0(Rb)

then Rx is never actually loaded. Consequently, the subsequent SUB Rz, Rz, Rx would not have the correct value in Rx. This example shows that when the loaded register is used more than once, the optimization cannot be applied safely.

Final Answer

1. Approximately 21% of the loads must be eliminated**.**

This is derived by requiring that the 5% slower clock cycle is compensated by reducing the dynamic instruction count by eliminating 0.228 \times p fraction of instructions, leading to $p \geq 0.2088p$.

2. Non-Replacement Example:

LOAD Rx, 0(Rb)
ADD Ry, Ry, Rx
SUB Rz, Rz, Rx

In this sequence, the load into Rx is used twice. Replacing the first two instructions would leave the later SUB without the correct value, so the transformation is not valid.

D1:

Modern RISC processors **still adhere to core principles** envisioned in the 1980s, even though their instruction sets have grown in number. Their evolution has been driven by practical needs such as support for multimedia, security and power efficiency; without sacrificing the fundamental simplicity that aids high performance pipelining and compiler optimization.

RISC instruction sets maintain simplicity since they adhere to basic design principles with a focus on efficiency and ease of implementation. For example, **fixed and simple instruction encoding**, where all instructions are uniform in length and format, decoding faster and supporting efficient pipelining. Another characteristic is **register-oriented operation** i.e. logical and arithmetic instructions operate primarily on registers, with memory references confined to special load and store instructions. This load/store architecture reduces the complexity in instruction execution. Additionally, the ISAs traditionally use **limited addressing modes**, avoiding the complex memory address schemes used by CISC architectures, making instruction execution even easier. Lastly, instruction sets apply **simple data types and operations** so that calculations are straightforward and efficient. These features combined are what make a RISC ISA simple despite how modern RISC processors are expanding their instruction sets.

D2:

The complexity of an ISA can be evaluated at two distinct levels its software interface versus its microarchitectural behavior. Each level has different implications:

Software Interface (Architectural Level)

Definition: This is the interface visible to compilers, assembly programmers, and operating systems. It includes the instruction set encoding, addressing modes and semantics.

Complexity Indicators: In x86, this is a very high-level of complexity. It has variable-length instructions, numerous addressing modes, and a very large but unbounded set of operations. All these add up to a CISC description.

Implications: Software tools have to deal with this unpredictability; hence compiler design and instruction scheduling are challenging. Complexity at this level guarantees legacy compatibility and an immense number of high-level constructs.

Microarchitectural Level (Implementation Level)

Definition: This is where the instructions actually get executed by the processor. New x86 processors translate the complex CISC instructions to simpler, fixed-length micro-operations (micro-ops) more similar to RISC.

Complexity Indicators: These are implemented most RISC ideas: a lightweight, register-oriented set of instructions, fewer branch instructions, and uniform instruction encoding for micro-ops. Designs like these make pipelining and parallel execution possible.

Implications: The hardware leverages more linear, more predictable execution that can be optimized for speed and power efficiency. This hidden RISC-like layer of operation lies at the heart of high-performance delivery by modern Intel processors.

Are Modern Intel Processors RISC?
At the Software Interface: Intel's x86 remains a CISC ISA because software developers and compilers face a feature-rich, variable-length, and complex instruction set.

At the Microarchitecture Level: The internal design converts CISC instructions to RISC-like micro-ops, i.e., internally the processors leverage RISC design strengths—simplicity, consistency and pipelining simplicity.

Intel processors today are hybrids. They maintain a balance of CISC interface for backward compatibility and high functionality, but internally apply RISC principles to provide performance. Hence, it depends on where you view complexity whether to term them RISC or CISC.